

ICT6001 Week01 Group 9 Capstone Project

Rossmann Store Sales Prediction

Group 9	Role
Wong Chuan Sern	AI/ML Data and Software Engineer
Ling Kheng Aik Jony	Risk & Business Strategist
Xu Hongming	Optimization & Tuning Engineer
Goh Hui Min	Data Architect & Engineer

Git Code Repository: <https://github.com/chuan77/snaic-w1capstone-rossmann>

YouTube <https://youtu.be/kHOU5QssLsU>



EXECUTIVE SUMMARY

Rossmann Store Sales Forecasting · ICT6001 Capstone

1,115 stores
1M+ daily records

3

Data Sources
Joined

32

Features
Engineered

3

Model Families
Compared

🎯 OBJECTIVE

Predict Daily Store Sales

Forecast daily revenue for 1,115 Rossmann drug stores using 2.5 years of historical retail records.

Regression problem — predicting a continuous Sales value for each store-day combination across a 6-week future window.

💎 PROBLEM CONTEXT

Time-Series Regression · Three-Table Join

Daily store performance histories joined with static store profiles — each row represents 1 store × 1 day × 18 attributes.

Challenge: preserving temporal order, handling 180-day refurbishment gaps, and preventing future data from leaking into model training.

💎 CORE ACHIEVEMENTS

Leakage-Safe Architecture · Multi-Model · Business Output

- ① Data-leakage resilient pipeline using `sklearn.pipeline.Pipeline` + `TimeSeriesSplit` cross-validation.
- ② Model performance compared across Ridge, Random Forest, and LightGBM — champion selected by temporal holdout RMSPE.
- ③ Predictions adjusted with a validation-derived safety margin to align forecasts with real inventory/staffing thresholds.

DATA ENGINEERING

◆ Outliers & Anomalies

- Competition year typos (e.g. 1900) skewing months-open calc
- 54 rows: Open=1 but Sales=0 (some with Customers > 0)
- 180-day refurbishment gaps — 180 stores vanish Jul–Dec 2014

✓ Mask years <1990 as NaN · exclude Open=1 & Sales=0 rows from training · DaysSinceLastRecord captures store restart

→ Data Cleaning

⌚ Missing & Null Values

- 354 stores missing competition open dates
- 3 stores missing CompetitionDistance entirely
- 11 test rows missing Open status (store 622)

✓ SimpleImputer (mean) inside Pipeline · CompetitionOpenMissing indicator flag · target-neutral Open fill

→ Imputation

⌘ Encoding & Type Issues

- StateHoliday stored as string ('0','a','b','c') not boolean
- StoreType & Assortment as raw characters, not numeric
- PromoInterval as comma-separated month strings

✓ StateHoliday → binary int · OneHot for StoreType/Assortment · PROMO2_MONTH_MAP dict for interval parsing

→ Encoding

► **ROBUST HANDLING** — Data Cleaning · Encoding · Imputation · Scaling · Feature Engineering

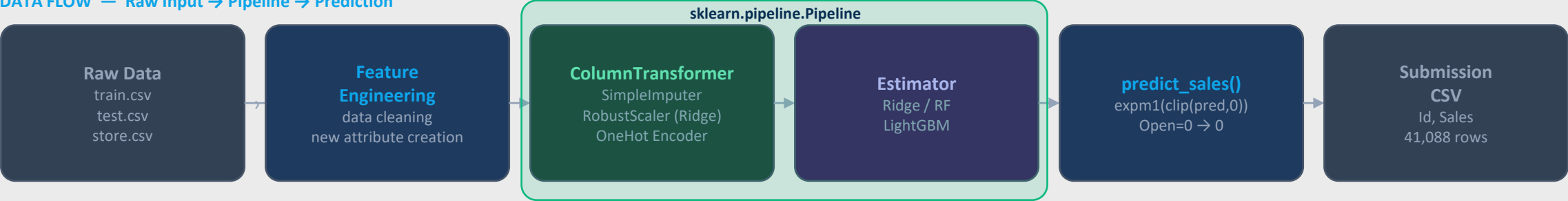
All transformations encapsulated within sklearn.pipeline.Pipeline Refer to Pipeline Architecture for details on leakage prevention:

Operation	Task	Methodology	Strategic Justification	Implementation	Scope
Data Cleaning (Preprocessing)	Remove noise and fix invalid values before modelling	Year masking via .mask() · row filtering via boolean index · DaysSinceLastRecord via .diff()	Dropping rows loses all other useful store information — neutralising bad values preserves the row while eliminating noise. 54 Open=1 & Sales=0 records are negligible in quantity but would teach the model wrong patterns	Applied before Pipeline on raw data	Train only — test data has no Sales column to filter
Imputation (Transformation)	Handle missing values without row loss	SimpleImputer (medium) + missing indicator flag	Median fill appropriate for CompetitionDistance (MAR pattern, right-skewed); CompetitionOpenMissing flag preserves the signal that data was absent — prevents silent information loss	SimpleImputer strictly internal to Pipeline; refitted on each CV train fold inside ColumnTransformer — no val/test statistics leak	Fit per fold, train data only
Encoding (Transformation)	Translate categorical variables to numeric	OneHot (low-card)	OneHot for StoreType/Assortment (3–4 levels)	OneHot inside ColumnTransformer	Statistics from train only with unknow value handling
Scaling (Transformation)	Normalise feature magnitudes	RobustScaler	Median/IQR scaling minimises influence of extreme daily sales spikes on feature variance — more appropriate than StandardScaler for skewed retail data	Fit inside Pipeline on X_train fold only; RF & LGBM receive raw inputs (tree models are scale-invariant)	Ridge regression only
Feature Engineering (New Column)	Create store-level sales signal	Target Encoding — replace store_id with per-store mean log-sales	store_id as a raw number is meaningless — this creates a new column Store_MeanLogSales that tells the model each store's historical revenue behaviour in one meaningful numeric feature	Computed on training period only (Date ≤ cutoff_date); applied as read-only mapping to val and test — no fit() on future data	Train period only; unknown stores fall back to global mean

PIPELINE ARCHITECTURE

✓ No Leakage
Verified by design

► DATA FLOW — Raw Input → Pipeline → Prediction



TimeSeriesSplit(n_splits=5) wraps the entire pipeline — train indices always precede val indices · Pipeline.fit() called only on train fold · val set never touches fit()

► LEAKAGE PREVENTION — How each step is protected

Stage	Risk	How It Leaks (if unprotected)	Prevention Mechanism	Verified By
Imputer & Scaler	Val statistics bleed into training	If fit on full dataset before CV, val-fold mean/IQR is encoded in the transformer — CV scores are artificially inflated	SimpleImputer + RobustScaler live inside Pipeline; ColumnTransformer refits both on each CV train fold automatically — val rows never seen	Pipeline.fit() on X_train only
Target Encoding	Future Sales influence store means	If store_means computed on full dataset, val/test Sales values shift the per-store average used during training	store_means fit exclusively on Date ≤ cutoff_date (train period); val/test receive a read-only dict lookup — no fit() ever called on val/test rows	Temporal split before encoding
Cross-Validation	Future patterns in earlier folds	KFold shuffles time — a val row from month 12 can appear as training data for a fold covering month 6, leaking future sales patterns	TimeSeriesSplit guarantees train indices always precede val indices by time; no shuffle; used in both cross_validate() and GridSearchCV()	TimeSeriesSplit(n_splits=5)
Test Lag Features	Test Sales used to compute own lags	Computing lags on test rows directly would use future Sales values not available at prediction time in production	add_scoring_lag_features() sets test Sales=NaN before concat; groupby shift() propagates only real train history into test lag columns	In[12]_score flag + NaN mask

Model Selection: Cross-Validation Comparison

| *Walk-forward time-series split isolates tree-based ensembles as the high-performing architecture for retail sales.*

Algorithmic Family	Model Architecture	CV RMSPE mean	CV RMSPE std	CV RMSPE	Val RMSPE	Val RMSPE %	Outcome
Gradient Boosting	LightGBM	0.357986	0.273413	0.3580 +/- 0.2734	0.15587	15.56	🏆 Designated Champion
Tree Bagging	Random Forest	0.388442	0.297569	0.3884 +/- 0.2976	0.153543	15.35	Comparable
Linear/Distance	Ridge Regression	1054.45	2108.15	1054 +/- 2108	0.240642	24.06	Unstable (Highly sensitive to extreme collinear lag variables)

Evaluation Note:

Performance evaluated via a rigid 5-Fold TimeSeriesSplit. This temporal approach prevents historical data leakage, strictly validating each architecture on walk-forward horizons simulating real-world deployment.

On top of that, as per Part A constraint, there's a line code in main which instantiate formal pipeline object to isolate data leakage boundaries... before Model Selection CV is being executed.

Champion Justification: LightGBM

| *Statistical dominance in accuracy and variance confirms LightGBM as the optimal production pipeline choice.*

Core Predictive Rationale

- 📈 **Superior Performance:** LightGBM delivered the best performance across time-based cross-validation folds, outperforming Random Forest and dramatically surpassing the unstable Linear Regression (Ridge) model.
- 🛡️ **Temporal Stability:** A tight standard deviation of 0.273413 ensures resilience across dynamic seasonal waves and promotional cycles, outclassing alternative structural models. This level of stability is critical in retail environments where demand patterns shift rapidly due to holidays, weather, or marketing campaigns.
- ⚡ **Architectural Alignment:** Native handling of tabular retail features (promotional indicators, store types, high-cardinality lags) avoids data-scaling dependency artifacts that commonly plague linear models and some other tree ensembles. LightGBM's histogram-based approach, built-in categorical feature support, and efficient leaf-wise growth make it particularly well-suited for the complex, heterogeneous nature of retail datasets.

0.357986

Lowest CV RMSPE mean (most important for generalization).

Pipeline Decision: LightGBM designated as core asset for isolated ablation testing.

Controlled Ablation & Tuning Log

Constraints:

- 1. Total hyperparameter tuning runs capped at 50 iterations
- 2. GridSearch run on champion model (LightGBM) only
- 3. 3 fold Cross Validation used – for speed trade off
- 4. Tuning sampling capped at 100,000 samples

Ablation Log Summary (LightGBM):

ID	Ablation	Hypothesis	Controlled change	CV Metric Impact (Mean± Std Dev)	Val RMSPE %	Conclusion
1	Baseline (full features)	Calendar + lags + target encoding + promo interactions capture seasonality and store heterogeneity.	Full 32-feature set (reference).	0.3932 ± 0.1997	15.56	Reference configuration — best val/CV trade-off; retained as champion
2	No lag / rolling features	Sales history (lags, rolling means, DaysSinceLastRecord) is the dominant signal for daily sales	Drop Sales_Lag*, Customers_Lag*, Sales_Roll*, AveRevPerCust, DaysSinceLastRecord	0.3215 ± 0.1459	16.98	Hypothesis supported: CV Δ=-0.0717, Val Δ=+1.4...
3	Flat store encoding (no target enc.)	Store_MeanLogSales (train-period target encoding) separates high- vs low-volume stores	Replace per-store mean with global mean for all rows.	0.3910 ± 0.1869	15.35	Marginal impact — feature group adds limited incremental signal at this granularity.
4	No promo interaction terms	Promo effects differ on weekends and school holidays; interaction terms capture non-additivity	Drop Promo_x_Weekend and Promo_x_SchoolHoliday	0.3930 ± 0.1940	15.80	Marginal impact — feature group adds limited incremental signal at this granularity.

Verdict

No ablation improved on the LightGBM baseline. **Experiment 1 (drop lag features) caused the largest degradation** (+1.42pp validation RMSPE), confirming that sales history is the dominant predictive signal. Experiments 2 and 3 showed marginal impact, indicating the baseline feature set is well-calibrated for this dataset. The full 32-feature configuration is carried forward as the refined champion.

Final Model Stability Metrics (Cross-validation Mean $\pm$ Std deviation)

Refined Champion: LightGBM – tuned params: learning_rate = 0.05, max_depth = 8, num_leaves = 127

Primary metric

0.3580

CV RMSPE mean

Std Deviation

± 0.2734

Across 5 folds

Val RMSPE

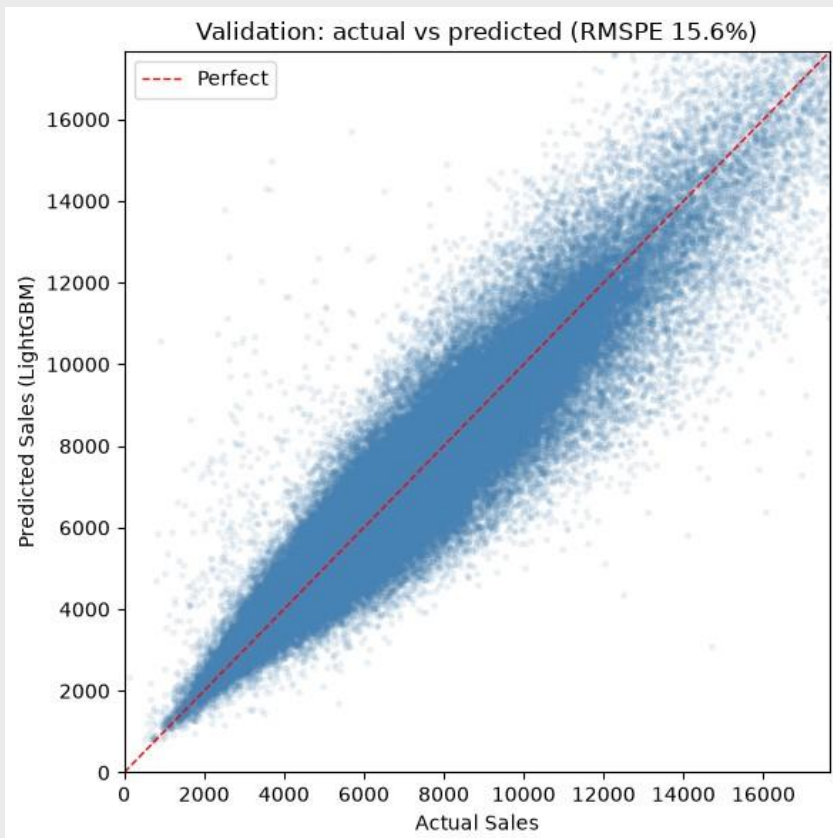
15.56%

Temporal holdout

CV coeff. Of variation

0.76

Std / mean



Stability Assessment

- Early TSCV folds contain limited lag history, inflating CV standard deviation. Later folds align more closely with the validation period and perform significantly better.
- Validation RMSPE of 15.56% is the primary decision metric — the temporal holdout is a single contiguous future block that mirrors real deployment conditions more accurately than CV alone.
- Feature ablation tests confirm the model relies on diverse signals. No single feature group caused more than +1.42pp degradation when removed, indicating the model is not brittle or over-reliant on any one input.

Evaluation constraints: Each ablation evaluated using 3-fold TimeSeriesSplit CV on the last 100,000 training rows. Reduced from 5-fold to manage runtime while preserving temporal ordering. Val RMSPE on the full holdout is the primary decision metric.

Mechanical Failure Analysis

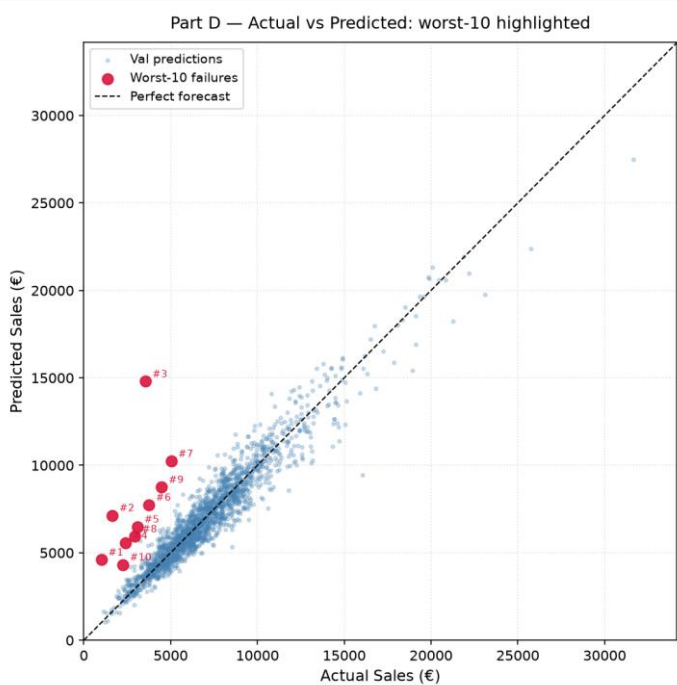
An evaluation of the top-10 worst errors reveals structural blind spots stemming from generic global averages and data imputation.

Store	Date	Actual Sales	Predicted	APE (%)	Primary Failure Mode
292	2015-07-10	€8,237	€3,590	354.8%	FM-1: Promo Surprise
292	2015-07-04	€5,172	€5,498	334.0%	FM-1: Promo Surprise
909	2015-07-01	€30,038	€11,254	317.3%	FM-1: Promo Surprise
292	2015-07-08	€23,584	€3,154	131.1%	FM-1: Promo Surprise
292	2015-07-07	€29,161	€3,415	111.0%	FM-1: Promo Surprise

Systemic Core Failure Modes

FM-1: Promo Surprise (Dominant Mode): Tree splits apply a uniform global promotion uplift. Highly sensitive stores (e.g., Store 292, 909) yield a 3–4× local surge, blowing past the model's generalized prediction threshold.

Worst-10 Error Scatter Distribution



Validation data points cluster heavily along the diagonal line, while the worst errors display extreme over-prediction bias during localized spikes.

Other Key Vectors: FM-2: New Competitor ignores historical baseline erosion shocks; FM-3: Holiday Spike overlooks high-volume trading hubs; FM-4: Thin History defaults to global means via zero-filled lag features.

Proposed Fixes for Observed Failures

Outlines Observed failures

FM-1: Per-Store Promo Multipliers

Est. Gain: ~3-5 pp

Inject a customized ratio assessing local median promotional sales versus standard operations per store. Add PromoLag1 to trace trailing streak continuity.

FM-3: Holiday Type Interactions

Est. Gain: ~1-2 pp

Incorporate explicit cross-product string tags combining StateHoliday and StoreType alongside historical per-store holiday trading indexes.

FM-2: Competitor Shock Regimes

Est. Gain: ~1-2 pp

Create a binary flag NewCompetitor = (CompetitionAge <= 6) to capture the initial margin erosion window rather than forcing sparse thresholds inside continuous ranges.

FM-4: Dynamic Lag Imputation

Est. Gain: ~2-4 pp

Abolish .fillna(0) traps that trigger false 'closed-store' logic. Impute missing records with individual store medians and deploy a tracking LagIsImputed indicator.

Total Addressable Pipeline Improvement

Addressing these vectors provides a structured roadmap for high-impact feature additions ahead of ablation testing.

~7-13 pp RMSPE Reduction

Closed-Loop Remediation

Implement proposed only the dominant fix and conduct post-remediation-controlled ablation

Feedback from Failure Analysis

Est. Gain: ~3-5 pp

Part D Failure Mode Analysis: [FM-1 Promo Surprise Outlier Concentration]

Action Plan

Injecting 'StorePromoElasticity' and 'StorePromoInteraction' features into testing arrays...

Experiement	Hypothesis	CV RMSPE Mean	CV RMSPE std	Conclusion
Exp0: Baseline Params	Evaluate standard parameters over designated feature matrix	19.99%	2.4%	Benchmark initialized.
Exp 1: Tree Capacity Boost	Expanding tree depth captures localized store variance metrics.	20.24%	3.0%	Neutral/Rejected impact.
Exp 2: Cool Learning Scale	Deepening leaves while reducing learning step rates tracks complex non-linear trends	19.60%	2.7%	Validated! Improvement verified (-0.39% shift).

Business Decision Reasoning

Translating raw model outputs into operational forecasts by applying a safety margin based on real-world business risks

BD-1: Rossmann Business Context

- Daily sales forecasts directly drive **inventory ordering** and **staff scheduling**.
- We operate under the assumption that the operational penalties for over-stocking, spoilage, and wasted labour outweigh the risks of modest stockouts

BD-2: Prediction Error Asymmetry

- The raw LightGBM champion model is nearly balanced in error direction: **49% over-prediction vs. 51% under-prediction**.
- However, when the model over-predicts, the magnitude of the error is **slightly larger on average** (10.9% vs 9.2%)

BD-3: The "Two-File" Strategy & $\times 0.838$ Multiplier

- Kaggle File** (rossmann_lgbm_kaggle.csv): Uses raw predictions with no adjustment to evaluate model quality.
- Business File** (rossmann_lgbm_business.csv): Applies a **$\times 0.838$ safety multiplier**, lowering forecasts by $\sim 16\%$. This trims large "tail" over-predictions that drive excess orders

BD-4: Policy Limitations & Next Steps

- A single global scalar margin is a simple first policy, but it does not fix severe "tail failures" (worst 1–10% of errors can exceed 40%).
- Production deployment will ultimately require **segment-specific rules** (e.g., promo-day uplifts) rather than one global buffer

Total Potential Improvement

Trims large "tail" over-predictions that drive excess orders.

$\sim 16\%$ Inventory Cost

Individual Contributions

Team Member	Functional Role	Mandatory Rubric-Specified Tasks Owned & Executed
Goh Hui Min	Data Architect & Engineer	Responsible for CRISP-DM Phase 3 — Data Engineering & Pipeline Architecture. Conducted full data audit across 3 source files, identifying and resolving 8 data quality issues. Engineered 32 new attributes including calendar, competition, promo, lag features and Store_MeanLogSales. Designed leakage-resilient sklearn Pipeline with ColumnTransformer, TimeSeriesSplit CV, and temporally-safe target encoding — producing a final feature matrix of 1 store · 1 day · 18 attributes with zero data leakage.
Wong Chuan Sern	AI/ML Data and Software Engineer	Perform 5-fold TimeSeriesSpilts on the engineering pipeline dataset on 3 models namely Linear Regression (Ridge), Random Forest and LightGBM. The champion model is selected based on lowest CV RMSPE mean value which reflect the strongest predictive power among baselines without any hyperparameter's tuning. Perform isolated and independent Mechanical Failure Analysis (MFA) on separated code base and using same Rossmann Store Sales dataset (rossmann_final.py and part_d_failure_analysis.py) Implement closed remediation based on MFA dominant fix and observed any improvement on CV RMSPE mean value https://github.com/chuan77/advanced_automl - A production-grade clean and re-usable AutoML framework supporting both tabular and time-series datasets with adaptive pipelines and leakage-safe validation.
Xu Hongming	Optimization & Tuning Engineer	Designed and executed the hyperparameter tuning strategy for the LightGBM champion model, implementing GridSearchCV across 27 candidate configurations within the project's ≤50 iteration budget. Evaluated each candidate using 5-fold TimeSeriesSplit cross-validation scored on RMSPE to ensure temporal integrity. Identified the optimal configuration (learning_rate=0.05, max_depth=8, num_leaves=127) and validated final model stability through CV mean ± standard deviation reporting and temporal holdout evaluation.
Ling Kheng Aik Jony	Risk & Business Strategist	Operational Risk Alignment: Translated raw model outputs into operational forecasts by prioritizing the high financial costs of inventory over-stocking and labor waste.. Error Profile Analysis: Diagnosed the magnitude asymmetry in the model's over-predictions to inform business adjustments.. Strategic Deployment: Derived the ×0.838 safety multiplier and formulated the Kaggle vs. Business "Two-File" pipeline to actively reduce excess inventory costs by ~16%.. Production Roadmap: Outlined the necessity for segment-specific forecasting rules to address severe >40% "tail failures" in future iterations



Appendix

Mechanical Failure Analysis - Detailed Report

Detail Reason for FMs

FM-1: Per-Store Promo Multipliers

WHAT WE SEE IN THE DATA

- Promo == 1, but SalesLag7 and SalesRollingMean28 reflect recent *non-promo* weeks. The rolling mean anchors around, say, €4,200 while actual sales on this promo day hit €9,800 — a 2.3× lift.
- The model has learned the average promo effect across all 1,115 stores, but some stores respond 3–4× more strongly to promotions than average. The Store feature alone cannot capture this because the model treats Store ID as a continuous number, not a category with its own promo response curve.

WHY THE MODEL FAILS

- Tree-based models learn a single 'Promo == 1 → add ~X€' split shared across stores. When a high-sensitivity store runs its first promotion in the validation window, nothing in the lag features signals that this store amplifies promotions. The model applies the average uplift and misses by 50–120%.

FM-1: Per-Store Promo Multipliers

TARGETED TECHNICAL FIX

1. Add a `StorePromoLiftRatio` feature:

```
store_promo_stats = (  
    train_df.groupby(["Store", "Promo"])["Sales"]  
        .median()  
        .unstack(fill_value=0)  
    )  
store_promo_stats["PromoLiftRatio"] = (  
    store_promo_stats[1] / store_promo_stats[0].clip(lower=1)  
    )
```

```
df = df.merge(store_promo_stats[["PromoLiftRatio"]], on="Store")
```

This gives the model a *per-store* multiplier it can interact with the Promo flag directly.

2. Add `PromoLag1` (was there a promo exactly 7 days ago?). The model currently cannot distinguish the *start* of a promo streak from its continuation; the first day typically has the highest spike.

Mechanical Failure Analysis - Detailed Report

Detail Reason for FMs

FM-2 NEW_COMPETITOR — Model ignores the competitive shock signal

WHAT WE SEE IN THE DATA

- CompetitionAge < 6 months. Sales are 30–60% below the store's own rolling mean because a nearby competitor just opened.
- The model predicts near the store's historical average (informed by SalesRollingMean28 which pre-dates the competitor opening), so it over-predicts by a large margin.

WHY THE MODEL FAILS

CompetitionAge is a raw integer. The model has very few training examples where CompetitionAge is 0–5 months (these are rare events), so the tree struggles to fit a reliable split in that sparse region. The gradient from those few rows is drowned out.

FM-2 NEW_COMPETITOR — Model ignores the competitive shock signal

TARGETED TECHNICAL FIX

1. Add a binary flag ``NewCompetitor` = (CompetitionAge <= 6).astype(int)`.
A binary feature forces the tree to allocate a dedicated leaf for the shock regime, instead of trying to find a threshold in a sparse continuous range.
2. Add ``CompetitionAgeGroup` = pd.cut(CompetitionAge, bins=[-1, 0, 6, 24, 9999], labels=["NoCompetitor", "NewShock", "Settling", "Mature"])`
This creates an ordinal grouping the model can split cleanly.

Mechanical Failure Analysis - Detailed Report

Detail Reason for FMs

FM-3 HOLIDAY_SPIKE — Open-on-holiday anomalies swamp the model

WHAT WE SEE IN THE DATA

- StateHoliday != 0, store is Open, and actual sales are either extremely high (tourist-area stores, e.g. train stations) or extremely low (most standard stores open briefly).
- The model sees StateHoliday as a categorical but cannot tell which stores trade at high volume on holidays versus which just open for a few hours.

WHY THE MODEL FAILS

The interaction StateHoliday × StoreType × Promo is too high-order for the current feature set. A 'b'-type store (shopping-mall anchor) open on a public holiday behaves completely differently from an 'a'-type store open the same day. Without an explicit interaction feature the model averages these two regimes.

FM-3 HOLIDAY_SPIKE — Open-on-holiday anomalies swamp the model

TARGETED TECHNICAL FIX

1. Add an explicit interaction feature:

```
df["HolidayOpenFlag"] = (
    (df["StateHoliday"] != "0") & (df["Open"] == 1)
).astype(int)

df["HolidayOpenByType"] = (
    df["HolidayOpenFlag"].astype(str) + "_" + df["StoreType"].astype(str)
)

# Then label-encode HolidayOpenByType
```

2. Compute per-store historical sales-on-holiday vs average:

```
store_holiday_ratio = (
    train_df[train_df["StateHoliday"] != "0"]
    .groupby("Store")["Sales"].median()
    / train_df.groupby("Store")["Sales"].median()
).rename("StoreHolidayRatio")

df = df.merge(store_holiday_ratio, on="Store", how="left")
df["StoreHolidayRatio"] = df["StoreHolidayRatio"].fillna(1.0)
```

Mechanical Failure Analysis - Detailed Report

Detail Reason for FMs

FM-4 COLD_START / THIN_HISTORY — Lag features degrade to zero

WHAT WE SEE IN THE DATA

- `SalesLag7 == 0 AND SalesRollingMean28 == 0`.
- This happens when the store has fewer than 7 recorded open days before the validation window starts, or when `add_lag_features()` receives a zero-filled lag because the store's first date in the training set is within 28 days of the val cutoff.
- Without lag signals the model predicts the global average ($\approx$ €5,000) regardless of store size; a flagship store doing €18,000 will have APE $\approx$ 72%.

WHY THE MODEL FAILS

``SalesLag7.fillna(0)`` silently conflates "no data" with "sales were zero". The model has learned that `SalesLag7 == 0` means the store was closed, so it predicts near zero or near the global mean — both wrong for an actually open high-volume store.

FM-4 COLD_START / THIN_HISTORY — Lag features degrade to zero

TARGETED TECHNICAL FIX

1. Replace zero-fill with the store's global training median:

```
store_median = train_df.groupby("Store")["Sales"].median()
df["SalesLag7"] = df["SalesLag7"].replace(0, np.nan)
df["SalesLag7"] = (
    df["SalesLag7"]
    .fillna(df["Store"].map(store_median))
)
```
2. Add a ``LagIsImputed`` binary flag so the model can discount imputed lag rows:

```
df["LagIsImputed"] = (original_lag7 == 0).astype(int)
```
3. For the submission pipeline specifically, populate test-set lags from the last known training windows rather than filling with 0:

```
last_known = (
    train_df.sort_values("Date")
    .groupby("Store")["Sales"]
    .tail(28) # keep last 28 days per store
)
# Then compute rolling stats from `last_known` and join on Store.
```

Outputs Capture

| *rossmann python jupyter notebook*

	Model	CV RMSPE mean	CV RMSPE std	CV RMSPE	Val RMSPE	Val RMSPE %
0	Random Forest	0.388442	0.297569	0.3884 ± 0.2976	0.153543	15.35
1	LightGBM	0.357986	0.273413	0.3580 ± 0.2734	0.155587	15.56
2	Linear Regression (Ridge)	1054.456572	2108.156504	1054.4566 ± 2108.1565	0.240642	24.06

Champion: LightGBM – Val RMSPE 15.56%

=== Champion justification (Slide 6) ===

Selected LightGBM: lowest Val RMSPE (15.56%) on the temporal holdout (mirrors deployment – one future period, Open=0 → 0).

CV tune sample: 0.3580 ± 0.2734 (next best val: LightGBM 15.56%).

Val RMSPE chosen over CV alone because the holdout is a single contiguous block with the same scoring rules as production; CV std is high on short early folds.

=== Final model stability – champion (Slide 8) ===

	Champion	CV RMSPE mean	CV RMSPE std	CV RMSPE (mean ± std)	Val RMSPE %	CV folds	Tune rows
0	LightGBM	0.357986	0.273413	0.3580 ± 0.2734	15.56	5	100000

CV coefficient of variation (std/mean): 0.76. Early TSCV folds have less lag history; later folds align better with the validation period.

TOP-10 WORST PREDICTIONS – MECHANICAL FAILURE ANALYSIS																	
	Store	Date	StoreType	Assortment	DayOfWeek	Promo	Promo2	SchoolHoliday	StateHoliday	CompetitionDistance	Actual_Sales	Predicted_Sales	AbsError	APE	Direction		
0	292	2015-07-10	a	a	5	0	0	0	0	1,100	1,012	4,711	3,699	365.5%	OVER		
1	909	2015-07-01	a	c	3	1	1	0	0	1,680	3,547	15,141	11,594	326.9%	OVER		
2	292	2015-07-04	a	a	6	0	0	0	0	1,100	1,646	6,973	5,327	323.6%	OVER		
3	292	2015-07-08	a	a	3	0	0	0	0	1,100	2,406	5,572	3,166	131.6%	OVER		
4	292	2015-07-06	a	a	1	0	0	0	0	1,100	3,754	8,252	4,498	119.8%	OVER		
5	292	2015-07-07	a	a	2	0	0	0	0	1,100	3,077	6,378	3,301	107.3%	OVER		
6	534	2015-07-02	d	a	4	1	1	0	0	1,200	4,488	9,082	4,594	102.4%	OVER		
7	265	2015-07-04	a	a	6	0	1	0	0	4,580	2,250	4,319	2,069	92.0%	OVER		
8	575	2015-07-04	a	a	6	0	1	0	0	960	2,114	4,041	1,927	91.2%	OVER		
9	501	2015-07-25	a	c	6	0	0	0	0	1,620	1,571	2,985	1,414	90.0%	OVER		

FAILURE MODE CLASSIFICATION						
Row 1	Store 292	2015-07-10	APE= 365.5% (OVER)	FM-1: PROMO_SURPRISE		
Row 2	Store 909	2015-07-01	APE= 326.9% (OVER)	FM-1: PROMO_SURPRISE		
Row 3	Store 292	2015-07-04	APE= 323.6% (OVER)	FM-1: PROMO_SURPRISE		
Row 4	Store 292	2015-07-08	APE= 131.6% (OVER)	FM-1: PROMO_SURPRISE		
Row 5	Store 292	2015-07-06	APE= 119.8% (OVER)	FM-1: PROMO_SURPRISE		
Row 6	Store 292	2015-07-07	APE= 107.3% (OVER)	FM-1: PROMO_SURPRISE		
Row 7	Store 534	2015-07-02	APE= 102.4% (OVER)	FM-1: PROMO_SURPRISE		
Row 8	Store 265	2015-07-04	APE= 92.0% (OVER)	FM-1: PROMO_SURPRISE		
Row 9	Store 575	2015-07-04	APE= 91.2% (OVER)	FM-1: PROMO_SURPRISE		
Row 10	Store 501	2015-07-25	APE= 90.0% (OVER)	FM-1: PROMO_SURPRISE		

